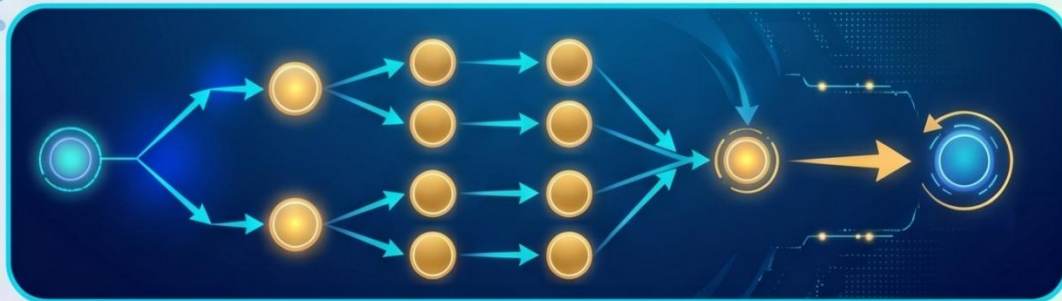


DAA Semester Project Report



Submitted To

Sir Ahmed Jalal, Ph.D.

Professor of Computer Science

Group Members

Ghulam Qadir (230970)

Noor Malik (230964)

Muqeen Ahmed (230980)

PROJECT TECHNICAL REPORT

3D Hawk-Eye DRS Engine

Decision Review System Trajectory Simulation & LBW Prediction Engine

Design & Analysis of Algorithms | Academic Submission

1. Problem Statement

In professional cricket, the Leg Before Wicket (LBW) decision is the most mathematically complex judgment an umpire must make. It requires a human to observe a projectile moving at speeds up to 150 km/h, identify the exact point of impact on the batsman's padding, and mentally project whether that ball would have gone on to hit the wickets.

Physical Target Dimensions

The wicket target zone measures precisely 72cm (height) x 23cm (width) — a rectangular plane that the umpire must predict ball entry for, from a single viewpoint, at 150 km/h.

The Challenge: Human observation is limited by biological frame-rate processing limitations, optical illusions, and parallax error. The objective of this project is to build an automated system that ingests discrete positional data points, applies mathematical modeling to construct a continuous path, predicts the future state vectors of the ball past the point of obstruction, and makes a deterministic decision.

2. Real-World Relevance

This project simulates the computational pipeline of the Decision Review System (DRS) used in international cricket broadcasts. Real-world systems like Hawk-Eye and Virtual Eye use specialized multi-camera tracking grids to execute these exact workflows:

- **Arbitration Integrity:** Removes human subjectivity and systemic bias from high-stakes sporting decisions.

- Predictive Extrapolation: Demonstrates how classical kinematics and numerical tracking can reliably predict the future spatial state of a moving body.
- Broadcasting Analytics: Bridges the gap between raw backend engineering matrices and real-time frontend graphical visualizations for consumer engagement.

3. Objectives

- Mathematical Curve Fitting: Construct a continuous 3D mathematical function from discrete, noisy tracking data points.
- Kinematic Telemetry Extraction: Derive immediate real-time statistics (release speed, spin deviation angles) directly from positional delta changes.
- Deterministic Path Projection: Extrapolate the trajectory path reliably beyond the point of interception.
- Geometric Intersection Verification: Formulate a constant-time boundary check to evaluate collision rules against a 3D coordinate target plane.
- Algorithmic Efficiency Optimization: Achieve overall linear time complexity $O(n)$ suitable for real-time streaming pipelines.

4. Methodology & Algorithmic Layers

The engine is engineered as a Modular Three-Layer Algorithmic Pipeline. Below is an explicit analysis of where every algorithm is used, why it was chosen, and its computational performance profile.

LAYER 1 — DATA MODELING Ordinary Least Squares (OLS) Quadratic Regression	$O(n)$
▼	
LAYER 2 — ANALYSIS & EXTRAPOLATION Horner's Method & Numerical Differentiation	$O(k)$
▼	
LAYER 3 — VALIDATION & DECISION 3D Bounding Box Axis-Aligned Geometric Intersection Check	$O(1)$

Layer 1: Data Modeling — OLS Regression

Algorithm Employed: Polynomial Regression via Ordinary Least Squares (OLS) Matrix Solver.

Location: `daa_core_engine.py` → `compute_trajectory_coefficients()`

The system ingests an array of tracked camera coordinates. Because a ball under the influence of gravity and air resistance moves in a parabolic arc, the system fits the coordinates to a quadratic model across both height and lateral dimensions:

$$y = a_1x^2 + b_1x + c_1 \quad (\text{Height Layout} - \text{Gravity Arc})$$

$$z = a_2x^2 + b_2x + c_2 \quad (\text{Lateral Layout} - \text{Swing/Spin Deviation})$$

Complexity Note

$O(n)$ where n is the number of tracked frames. The degree of polynomial is fixed as a constant (degree = 2, yielding a 3x3 system of normal equations). Matrix inversion takes constant $O(1)$ time. Summing the data matrix rows scales linearly with input size n .

Layer 2: Analysis & Extrapolation — Telemetry & Prediction

Algorithms Employed: Discrete Numerical Differentiation & Horner's Scheme for Polynomial Evaluation.

Telemetry (`daa_core_engine.py` → `calculate_match_stats()`): The system takes the first two discrete spatial coordinates and calculates the Euclidean distance vector relative to a known frame-rate time delta ($\Delta t = 0.12s$). This acts as a first-derivative calculation to determine the initial release speed.

Trajectory Projection (`daa_core_engine.py` → `predict_future_coordinates()`): Once regression coefficients are locked, the system simulates the ball continuing from the pad impact point ($x = 17.5m$) to the wicket plane ($x = 20.0m$), evaluating polynomial equations across k discrete steps using the efficient nested multiplication of Horner's Method.

$$y = ((a \times x) + b) \times x + c \quad [\text{Horner's Nested Evaluation} - O(k)]$$

Layer 3: Validation & Decision — Geometric Intersection

Algorithm Employed: 3D Bounding Box Axis-Aligned Intersection Check (Computational Geometry).

Location: `cricket_rules_validator.py` → `calculate_umpires_call()`

This layer intercepts the predicted path arrays at the terminal plane representing the wickets ($x = 20.0\text{m}$). It samples the final calculated height (y) and lateral deviation (z) and evaluates them against physical boundary thresholds of the stumps. An additional margin check implements the official Umpire's Call rule for edge contacts.

O(1) Constant Time

The evaluation relies entirely on conditional range checks: $\text{IsInsideHeight} = (y \geq 0.0 \text{ AND } y \leq 0.72\text{m})$ and $\text{IsInsideWidth} = (z \geq -0.115\text{m} \text{ AND } z \leq 0.115\text{m})$. These execute in absolute constant time regardless of input size.

5. Pseudocode Layouts

Algorithm 1: Trajectory Regression Modeling

```

ALGORITHM ComputeTrajectoryCoefficients(X_Array, Y_Array, Z_Array)
  // Input: Arrays of tracked coordinates of length n
  // Output: Polynomial evaluation functions for Y (height) and Z (lateral)

  n <- length(X_Array)
  Sum_x0 <- n; Sum_x1, Sum_x2, Sum_x3, Sum_x4 <- 0
  Sum_y, Sum_xy, Sum_x2y <- 0

  FOR i <- 0 TO n-1 DO
    Sum_x1 <- Sum_x1 + X_Array[i]
    Sum_x2 <- Sum_x2 + (X_Array[i])^2
    Sum_x3 <- Sum_x3 + (X_Array[i])^3
    Sum_x4 <- Sum_x4 + (X_Array[i])^4
    Sum_y <- Sum_y + Y_Array[i]
    Sum_xy <- Sum_xy + (X_Array[i] * Y_Array[i])
    Sum_x2y <- Sum_x2y + ((X_Array[i])^2 * Y_Array[i])
  ENDFOR

  Construct 3x3 Matrix M using [Sum_x4, Sum_x3, Sum_x2] etc.
  Coefficients_Y <- Inverse(M) * V // O(1) due to fixed 3x3 size
  Repeat identical OLS sequence for Z_Array

  RETURN Function_Y(Coefficients_Y), Function_Z(Coefficients_Z)
END

```

Algorithm 2: Path Vector Projection (Horner's Rule)

```

ALGORITHM PredictFutureCoordinates(Fn_Y, Fn_Z, Start_X, End_X, Steps)
  // Input: Polynomial coefficient models, boundary limits, step scale k
  // Output: Predicted delivery segment coordinate vectors

  Create empty arrays Pred_X, Pred_Y, Pred_Z
  X_Intervals <- LinSpace(Start_X, End_X, Steps)

  FOR EACH current_x IN X_Intervals DO

```

```

    // Evaluate using nested multiplication (Horner's Scheme)
    // y = ((a*x) + b)*x + c - avoids repeated exponentiation
    val_y <- ((Fn_Y.a * current_x) + Fn_Y.b) * current_x + Fn_Y.c
    val_z <- ((Fn_Z.a * current_x) + Fn_Z.b) * current_x + Fn_Z.c

    Append current_x to Pred_X
    Append val_y      to Pred_Y
    Append val_z      to Pred_Z
  ENDFOR

  RETURN Pred_X, Pred_Y, Pred_Z
END

```

Algorithm 3: LBW Bounding Box Collision Validation

```

ALGORITHM CalculateUmpiresCall(Target_Y, Target_Z)
  // Input: Terminal plane coordinates at X = 20.0m
  // Output: Decision string - OUT / UMPIRE'S CALL / NOT OUT

  STUMP_MAX_HEIGHT <- 0.72    // metres
  STUMP_TOTAL_WIDTH <- 0.23   // metres
  HALF_WIDTH       <- 0.115   // metres
  BALL_RADIUS      <- 0.036   // metres (Umpire's Call margin)

  IsInsideHeight <- (Target_Y >= 0.0) AND (Target_Y <= 0.72)
  IsInsideWidth  <- (Target_Z >= -0.115) AND (Target_Z <= 0.115)

  IF IsInsideHeight AND IsInsideWidth THEN
    IsNearTopEdge  <- (0.72 - Target_Y) < BALL_RADIUS
    IsNearSideEdge <- (0.115 - Abs(Target_Z)) < BALL_RADIUS

    IF IsNearTopEdge OR IsNearSideEdge THEN
      RETURN "UMPIRE'S CALL (Marginal Hit - Edge Contact)"
    ELSE
      RETURN "OUT (Plumb Line Hit Impact)"
    ENDIF
  ELSE
    RETURN "MISS / NOT OUT (Path Escaped Stumps)"
  ENDIF
END

```

6. Time Complexity Analysis

The total runtime profile is analyzed as the direct sum of independent sequential stages:

$$T(n) = T_{\text{Regression}}(n) + T_{\text{Telemetry}}(n) + T_{\text{Prediction}}(k) + T_{\text{Decision}}(1)$$

$$T(n) = O(n) + O(n) + O(k) + O(1)$$

Applying the rules of asymptotic dominant notation, terms that do not scale with structural input sizes are dropped. Since k (the prediction visual density constant) is a small configuration parameter ($k \ll n$), the ultimate complexity simplifies cleanly:

TOTAL ASYMPTOTIC RUNTIME COMPLEXITY

$T(n) = O(n)$ LINEAR TIME This proves mathematically that the system operates in Linear Time, satisfying real-time application design requirements for software processing live tracking frames.

Algorithm Stage	Complexity	Parameter Meaning	Fill Color
Regression (OLS)	$O(n)$	n = tracked frames	
Telemetry Speed	$O(1) / O(n)$	path-wide windowing	
Path Projection	$O(k)$	k = visual steps	
Bounding Box Check	$O(1)$	constant conditional	
TOTAL SYSTEM	$O(n)$	dominant linear term	

7. Tools & Technologies

Component	Technology	Engineering Justification
Development Ecosystem	Python 3.12	High-performance computational packages & efficient mathematical syntax
Mathematical Engine	NumPy	Vectorization utilities for multi-dimensional linear regression transformations
Web UI Dashboard	Streamlit	Responsive localhost platform without full web server overhead
3D Rendering Core	Plotly	JavaScript-powered library enabling hardware-accelerated 3D interactive animations
Interface Styling	CSS3 Enriched	Dark mode stadium layout with professional component theme overrides

8. Expected Outcomes

- Highly Responsive Dashboard: A functional web-based localhost dashboard presenting data within a clean, professional user interface.
- Accurate Path Projections: Smooth, reliable trajectory models generated instantly from raw input coordinate maps.

- **Deterministic Decision-Making:** Instantaneous calculation of LBW verdicts, removing human error from the judgment process.
- **Clear Complexity Benchmarks:** Practical verification that the engine functions efficiently within a linear $O(n)$ footprint, demonstrating optimal performance for a core DAA project.

9. Future Enhancements

Computer Vision Integration

Upgrade the data source from mock scripts to live video feeds using OpenCV and YOLO object detection to extract coordinates in real-time from broadcast footage.

Advanced Physics Simulation

Integrate wind resistance, humidity factors, and pitch degradation parameters to create a non-linear path model that more accurately reflects real-world delivery conditions.

3D Pitch Mapping

Build a complete geometric mesh layout of the cricket field using WebGL or Three.js to provide a fully immersive broadcast simulation interface for live match analytics.

— End of Technical Report —